13 An introduction to React Native

This chapter covers

- An overview of React Native
- Differences between React and React Native
- Ways to learn more about React Native

At this point, you've been over the basics of using React, implemented a router, explored Redux, looked at server-side rendering, and even transitioned to using React Router. What's left? There's still plenty to learn and explore in the React ecosystem and community. This chapter takes a high-level look at React Native, another project in the React ecosystem developed by Facebook. With React Native, you can write React applications that run on mobile platforms like iOS and Android. This means you can write applications that run on smartphones and any other platforms that React Native targets now or in the future. React Native provides an excellent developer experience when building these mobile applications in a React-like way, and this is a large part of why it's becoming increasingly important and popular in the React community.

Because React Native and getting started with mobile development encompass a substantially large domain, I'll keep our discussion of React Native concise and focused mainly on higher-level concepts. By the end of the chapter, you should

Licensed to Samir Mashlum <smashlum@gmail.com>

have an idea of what React Native is and why you might want to use it, and you'll know how to get started on learning more about it.

13.1 Introducing React Native

Before React Native came on the scene, you had a few options when it came to creating mobile applications. You could either use the iOS and Android platforms and languages available to you or you could opt for one of the hybrid approaches available. These vary in how they're implemented, but they would often utilize a web view (think "mobile browser") and expose some interfaces to the native SDKs. One downside to this approach is that although you can write native applications that allow you to use many familiar web APIs and idioms, the app wasn't "really native," and there would sometimes be a noticeable difference in performance and overall feel. The benefit was that teams or developers without expertise in mobile development could transfer their web-related skills and be able to create a mobile app.

The subject of mobile development and how platforms, languages, and hardware in this world all play their different roles is beyond the scope of this book. But the choice between hybrid and all-native approaches is relevant to our discussion of React Native because React Native offers a new alternative. With React Native, you can build apps that are "really native" but you can use a combination of JavaScript and platform-specific code (like Swift or Java).

React Native aims to bring the idioms and concepts of building user interfaces with React to mobile application development and blend the best aspects of mobile and browser development. It encourages code-sharing across platforms (there are components that target both iOS and Android devices), allows you to write native code where appropriate, and compiles to a native application—all while using many of the same idioms familiar to React.

Let's glance at a few top-level features of React Native:

- With React Native, you can write JavaScript applications that can also use native code (Swift or Java) and compile to native applications that run on iOS or Android.
- React Native can handle creating the same UI elements on Android and iOS, potentially simplifying the development of mobile applications.
- You can add your own native code when you need to, so you aren't constrained to using only JavaScript.
- React Native apps share idioms with React and provide the same component-driven, declarative concepts, and even APIs in some cases, to work with when designing your UI.
- The developer tooling for building React Native applications allows you to reload your application with changes without having to wait for a long compile cycle. This often saves developers time and can make for a more pleasant experience.
- The ability to share code and target multiple platforms can sometimes reduce the number of engineers dedicated to building a particular app or project. It

Licensed to Samir Mashlum <smashlum@gmail.com>

can lead to fewer codebases to maintain, and engineers can more easily move between web and native platforms.

- You can share logic and other aspects of React web apps with React Native apps, such as business logic and even styles in some cases.

How does React Native work? It may seem like a mysterious, black-box process for something to take your JavaScript and output a compiled, native application. You don't need to know how every part of React Native works in order to work with it, just like you don't need to know the ins and outs of React-DOM to write great React applications. But it's often helpful to have at least a working understanding of the technology you're using.

With React Native, you can create applications that are a blend of JavaScript and native code. React Native makes this possible by creating a bridge of sorts between your application and the underlying mobile platform. Most mobile devices can execute JavaScript, and React Native takes advantage of that to run your JavaScript. When your JavaScript is executed alongside any native code, React Native's bridging system uses the `React` core library, among others, to translate the component hierarchy (with event handlers, state, props, and styles) into a view on the mobile device.

When updates occur (for example, a user presses a button), React Native translates the native event (a press, a shake, a geolocation event, or whatever it is) into an event your JavaScript or native code can handle. It also renders the proper UI based on changes to state or props. React Native will also bundle all your code and do any necessary compilation so you can release your app to the Apple App Store or Google Play Store.

There's much more to these processes and to how React Native works, but the basic process of translating between the JavaScript running on the device and the native platform APIs and events is where the "magic" of React Native happens. The result is a platform that you can work with but that also doesn't compromise when it comes to performance. It's a happy medium between the problems of previous hybrid approaches to mobile apps, and it also avoids some of the pain points of traditional mobile development. Figure 13.1 illustrates an overall view of how it works.

If that sounds like a departure from the React you've been learning in this book, well, it is in many ways. But more important than the differences are the similarities. I'll cover those more in the next section, but you can look at the code in listing 13.1 to see how similar a React Native component is to the components you've worked with so far.

You can still see what the code in listing 13.1 does even though I'm not covering how to set up a React Native project in this chapter. Visit https://repl.it/KOAE/3 if you want to see what the code is doing and play around with React Native. Repl.it is an online platform for running and sharing code in an interactive way, and it has support for React Native. You'll be able to scan a QR code with your phone to view your React Native playground app. It's a great way to experiment with React Native without having to do any setup or configuration.

Licensed to Samir Mashlum <smashlum@gmail.com>

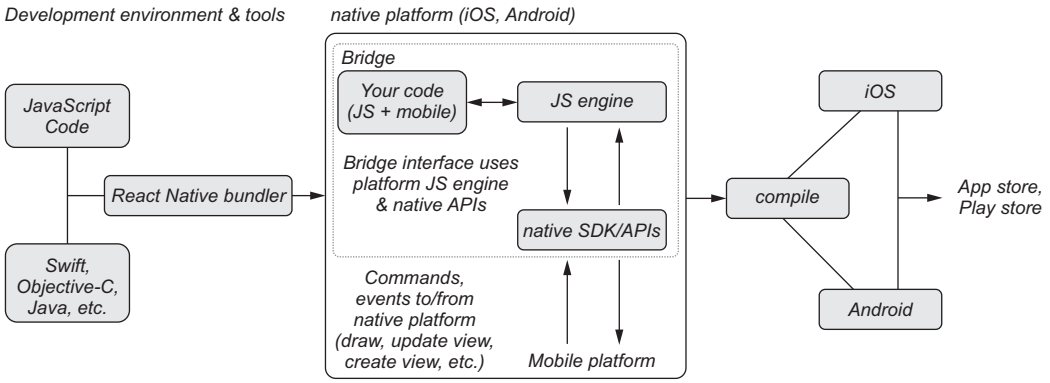


Figure 13.1 React Native works by creating a bridge between your JavaScript and the underlying native platform. Most native platforms implement a JavaScript virtual machine or other way of running JavaScript natively. The bridge enables execution of your application's JavaScript. The React Native bridging system will relay messages between the underlying platform and your JavaScript so that native events can be translated into ones your React components can understand and respond to.

One important thing you might notice is that the elements of the component (View, Text) are analogous to the `div` and `span` elements in your components from earlier chapters. This is an example of the broad React concepts persisting across platforms. It doesn't matter so much what the individual elements of a component are as much as that you can reuse and compose them, as shown in this listing.

Listing 13.1 React Native example component

```
import React, { Component } from 'react';
import { Text, View } from 'react-native';

export default class WhyReactNativeIsSoGreat extends Component {
    render() {
        return (
            <View>
                <Text>
                    If you like React on the web, you'll like React Native.
                </Text>
                <Text>
                    You just use native components like 'View' and 'Text',
                    instead of web components like 'div' and 'span'.
                </Text>
            </View>
        );
    }
}
```

You can still use regular React.Component, even in native app

React Native comes with basic ingredients to build mobile applications.

Text is more like span in browser

You can compose components with React Native; view component here is like a div in browser (common layout component)

There are other projects like React VR where the focus is even more divergent from the web UIs you've been working with but that use the same patterns and concepts.

Licensed to Samir Mashlum <smashlum@gmail.com>

This is one of the most powerful aspects of the React platform and is especially apparent when you see it across platforms. Learn more about React VR at https://facebook.github.io/react-vr.

13.2 React and React Native

How similar are React and React Native? Beyond sharing a name, they both use the `React` core library but target different platforms (browsers and mobile devices). This section will briefly look at some of their differences and similarities. Let's compare some of the important aspects of React and React Native:

- *Runtime*—React and React Native target different platforms. React targets browsers and thus heavily uses the browser-specific APIs. You can see some of the results of this in each API. For example, properties like class, ID, and others are commonly seen in web-based React components. Native platforms use different layout and styling semantics, so you won't see many of these properties on React Native components. Browser-based and mobile applications also run on different types of devices, so things like threading, CPU utilization, and other differences in the underlying technology shouldn't be ignored when thinking about React and React Native.
- *Core APIs*—Many of the React-specific APIs (like those used in component lifecycles, state, props, and so on) are similar across React and React Native. But each platform implements different APIs for networking, layout, geolocation, resource management, persistence, events, and other important areas. React Native aims to import some familiar APIs from the browser-oriented world, like the Fetch API for networking (https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API) and the Flexbox API for layout (https://developer.mozilla.org/en-US/docs/Web/CSS/flex). React Native also exposes events, but they're more specific to mobile platforms (`onPress`, for instance). These differences can be a minor hurdle, but fortunately there are libraries that help to eliminate differences between web and native APIs, like `react-primitives` (https://github.com/lelandrichardson/react-primitives).
- *Components*—The web-based React project doesn't have "built-in" components (for example, for images, text layout, or other UI elements). You create these yourself. React Native, on the other hand, does include components for things like text, views, images, and more. These are primitives you need to create UIs for mobile applications and are similar to what DOM elements are for browser environments.
- *Use of `React` core library*—Both React and React Native use the `React` core library for component definition. Each project utilizes a different rendering system to wire everything together and interact with the device (browser or mobile). React for web uses the `react-dom` library, whereas React Native implements its own system. This approach enables you to write components in similar ways across platforms.

Licensed to Samir Mashlum <smashlum@gmail.com>

- *Lifecycle methods*—React Native components also have lifecycle methods since they inherit from the same `React` base class, and those methods are also handled by the platform-specific system (React-DOM or React Native).
- *Event types*—Whereas React-DOM implements a synthetic event system that allows your components to work with browser events in a standard way, mobile applications expose other events. One example is gestures. You can pan, zoom, drag, and more on touch devices. Components written in React Native components allow you to respond to these events.
- *Styling*—Because React Native doesn't target browsers, you'll need to style your components in slightly different ways. There is no CSS API in regular mobile development, but you can use most CSS properties with React Native. React Native provides a specific API where 1:1 correspondence between properties isn't possible. Take CSS animations, for example. The CSS specification and the ways browsers implement it are different than how iOS and Android enable and implement animations, so you'll need to animate differently and use the right API for each platform. Learning new APIs for styling can take time and can prevent directly sharing CSS styling across web and native projects. Thankfully, though, there are libraries that work with React and React Native, like `styled-components` (www.styled-components.com). With the increasing popularity of React Native, you should expect to see more of these cross-platform libraries being developed.
- *Third-party dependencies*—As with React, you can still use third-party component libraries for React Native. Many popular libraries, like `React Router` and `styled-components`, even include variants that target React Native (as noted earlier). One of the most appealing aspects of React Native is that it can still take advantage of the JavaScript module ecosystem.
- *Distribution*—Although you can deploy React applications to almost any modern browser, React Native applications require platform-specific distribution tooling for both development and final release (Xcode, for example). You'll usually need to use the React Native build process to compile your application for final upload. The "walled garden" nature of the iOS and Android tooling is a well-known tradeoff to developing mobile applications.
- *Development tooling*—React for web runs in browsers, so you have the benefit of any browser-specific tools to help with debugging and development. For React Native, you aren't required to have the platform-specific tooling available, but it can still be useful. One key difference between the projects is that React Native has a focus on hot reloading that isn't part of React by default. *Hot reloading* can speed up mobile development because you don't have to wait for your app to compile. Figure 13.2 shows an example of some of the developer tools you get access to when working with React Native.

Licensed to Samir Mashlum <smashlum@gmail.com>

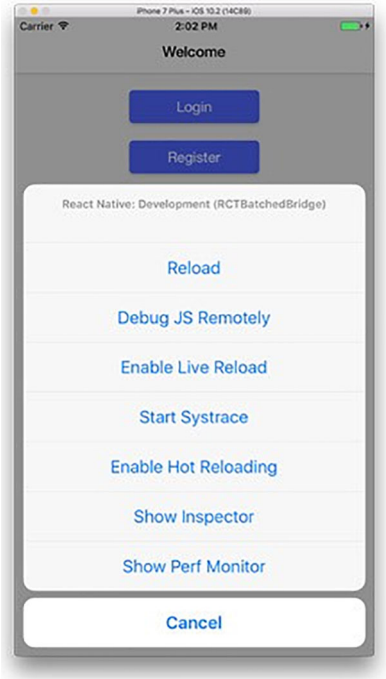


Figure 13.2 React Native comes with a number of additional developer tools that help with performance, debugging, and other functionality. These tools also mean you have less of a strict reliance on tools like Xcode for development, although you can certainly still use your platform-specific tools for development. Although there are many reasons, the excellent developer experience provided by React Native seems to be one reason why it's been received especially well as a technology.

13.3 When to use React Native

Not every developer and not every team has a need for React Native. Let's imagine a few scenarios you could find yourself in and see how React Native might or might not be something you should consider:

- *Solo developer*—If you're learning React for the first time or just using it for side projects, you'll probably learn React Native for fun or if you work on any mobile projects. React Native is also something to consider if you aren't deeply experienced with native development but want to ease into it or have a more straightforward app. If you already know React, it can make sense to dive into using React Native for mobile development with some familiar concepts at your disposal.
- *Small cross-functional team*—Small startups are often in a position where engineers will work on a broad slice of the stack, ranging from server to client applications (web, mobile, or otherwise). In situations like this, React Native can sometimes present a way for engineers who wear many hats for the organization to work on a mobile app without deep mobile experience and have their React momentum carry over. This could also apply to large organizations that want to easily move engineers between apps or projects.
- *Team with little to moderate native expertise*—If you or your team have little to moderate expertise with mobile development but are familiar with React and Java-

Licensed to Samir Mashlum <smashlum@gmail.com>

Script, React Native may make it easier for you to get your product together quickly. There's no substitute for experience, but not having to go all-in on Swift (iOS) or Java (Android) could potentially save you time.

- *Deep native expertise*—Some teams will choose React Native not because it lowers the barrier to mobile development in some ways, but because it helps standardize idioms and patterns across the various implementations of an app for a business (mobile and desktop). But if that's not a problem and you already have significant expertise and time invested in mobile development, React Native may need closer evaluation to see whether your team would benefit from the available abstractions and patterns.

Aside from the team and expertise considerations you might make when thinking about React Native, you should also be aware of some of the limitations that are inherent to the technology as it exists today:

- *Use of JavaScript*—If your team or organization doesn't have any JavaScript-focused developers or is already highly experienced with mobile development, it may not make sense to transition engineers to a JavaScript and JavaScript-focused ecosystem, and that's okay. Like React for web, React Native is not a silver bullet and should be evaluated based on tradeoffs, not the hype around it.
- *Specific performance needs*—React Native is performant, but as an abstraction it can present another barrier to achieving specific performance goals that you or your team might have. For example, if rendering 3D scenes is the primary goal of an application, React Native will probably not be the best fit. Other frameworks (like Unity) are probably better suited. This is in keeping with the "React is not a silver bullet" idea I just mentioned and that I've tried to maintain in earlier chapters.
- *Highly specialized app*—Some application types aren't a good fit for the React model. Augmented reality (AR), graphics-intensive, or other highly specialized applications often require special libraries and skills that most web engineers aren't equipped with. This isn't to say that it can't be done, but as of now React Native doesn't focus on addressing these needs.
- *Internal application*—Sometimes larger companies develop apps for internal use that help employees do their job better in a variety of ways. React Native can be well suited for these sorts of apps because such apps usually involve a relatively simple UI and can be iterated on quickly by engineers who don't specialize in mobile development.

Of course, it's ultimately up to you and your team to evaluate whether a technology makes sense for your use case, but hopefully you now have a better sense of when it might or might not make sense to use React Native.

Licensed to Samir Mashlum <smashlum@gmail.com>

13.4 The simplest "Hello World"

Even though I won't cover how to integrate React Native with Letters Social, this section spends a little time walking through a basic "Hello World" example so you can see it in action. You'll work outside the Letters Social repository, so feel free to place the app code wherever you like to keep track of code on your computer. Run the commands in the following listing to get started.

Listing 13.2 Installing `create-react-native-app`

```
cd ./path-to-your-react-native-sample-folder

npm install -g create-react-native-app

create-react-native-app .
```

After you've run these commands, you should be able to see a number of files created in your desired directory and some instructions. These commands are similar to those available in Create React App, a similar project focused just on React.js for the web platform. You can learn more about Create React App at https://github.com/facebookincubator/create-react-app. Figure 13.3 shows what you should see when getting started with the Create React Native App library.

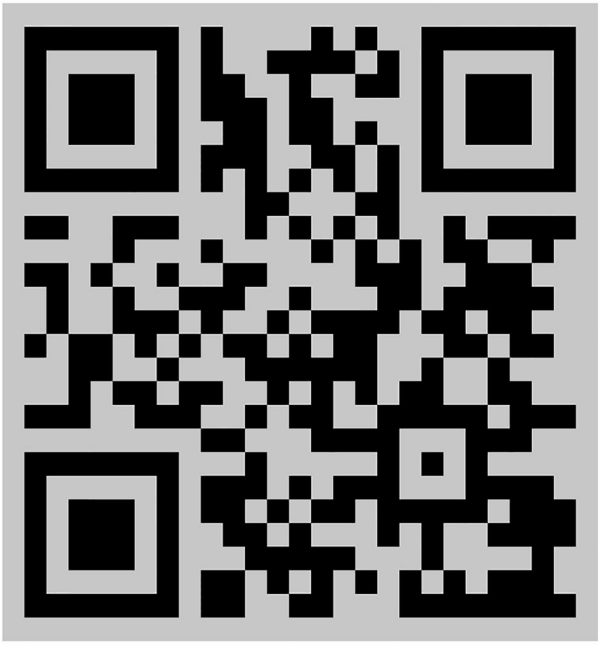
The Create React Native App tool installed dependencies, created some boilerplate files, set up a build process, and integrated the Expo React Native toolkit into the project. The Expo SDK extends React Native's functionality and makes working with the hardware technologies easier, among other things. The Expo XDE development environment makes it easy to manage multiple React Native projects as well as build and deploy them.

You won't build anything substantial, but you can tinker around and get a sense for how easy it might be to start building applications with React Native. Once you have the React Native packager running with `yarn start`, open one of the emulators (Android or iOS) so you can see the running app. Swap out some of the boilerplate code and see the hot reloading happening. Listing 13.3 shows a simple component that fetches some data from the Star Wars API when it's mounted. Notice that React Native is already using modern web APIs like Flexbox and Fetch (which you used a polyfill for in earlier chapters).

Licensed to Samir Mashlum <smashlum@gmail.com>

```
~/Code/oss/letters-native 6s
△ yarn start
yarn start v1.0.2
$ react-native-scripts start
00:16:28: Starting packager...
Packager started!

To view your app with live reloading, point the Expo app to this QR code.
You'll find the QR scanner on the Projects tab of the app.
```



```
Or enter this address in the Expo app's search bar:

  exp://10.0.1.5:19000

Your phone will need to be on the same local network as this computer.
For links to install the Expo app, please visit https://expo.io.

Logs from serving your app will appear here. Press Ctrl+C at any time to stop.

 › Press a to open Android device or emulator, or i to open iOS emulator.
 › Press q to display QR code.
 › Press r to restart packager, or R to restart packager and clear cache.
 › Press d to toggle development mode. (current mode: development)
```

Figure 13.3 When you start the application in development mode, you should see the React Native packager start and see a message like the one shown here. Follow the instructions to make sure you have the Expo XDE set up on your local machine. Depending on what environment you'd like to target, open either the Android or iOS simulator.

Licensed to Samir Mashlum <smashlum@gmail.com>

Listing 13.3 Simple React Native example (App.js)

```
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

export default class App extends React.Component {
    constructor(props) {
        super(props);
        this.state = {
            people: []
        };
    }
    async componentDidMount() {
        const res = await fetch('https://swapi.co/api/people');
        const { results } = await res.json();
        this.setState(() => {
            return {
                people: results
            };
        });
    }
    render() {
        return (
            <View style={styles.container}>
                <Text style={{ color: '#fcd433', fontSize: 40, padding: 10 }}>
                    A long time ago, in a Galaxy far, far away...
                </Text>
                <Text>Here are some cool people:</Text>
                {this.state.people.map(p => {
                    return (
                        <Text style={{ color: '#fcd433' }} key={p.name}>
                            {p.name}
                        </Text>
                    );
                })}
            </View>
        );
    }
}

const styles = StyleSheet.create({
    container: {
        flex: 1,
        backgroundColor: '#000',
        alignItems: 'center',
        justifyContent: 'center'
    }
});
```

Unlike React, React Native comes with primitive components for your UI.

Constructor, state initialization, and lifecycle methods are the same in React and React Native

You can use modern JavaScript features like async/await in React Native apps too.

Even though styles appear similar in React Native, you're not using CSS.

JSX expressions are the same in React Native and React

Creating a stylesheet in React Native requires use of its Stylesheet API to style your components.

If you make changes to the app, you should see the packager respond and update your running app in real time, as shown in figure 13.4. I hope this gives you a sense of how easy it can be to build applications in React Native. You may be used to hot reloading on the web, but for mobile development the compile-inspect-recompile cycle can take up a significant amount of time.

Licensed to Samir Mashlum <smashlum@gmail.com>

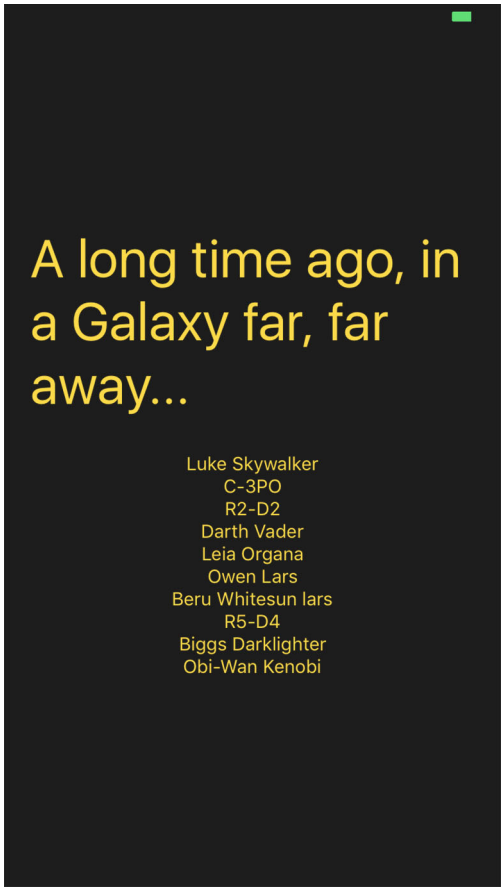


Figure 13.4 You should be able to see changes being instantly reflected in the simulator running your application code.

With that, you've created your first React Native component and code, which should give you a brief glimpse into how the technology works and how easy it can be to work with.

13.5 Where to go next

One of the phrases you'll see in the React docs, library ecosystem, and community is *learn once, write anywhere*. This is an homage of sorts to the *write once, run anywhere* phrase that's popular within the Java community and that is one of the hallmarks of the React paradigm. As we've seen in this chapter, you can learn React concepts and apply them to a variety of platforms, ranging from web to mobile to VR. There will be platform-specific differences and nuances whenever you learn how to use React on a new platform, but much of your React knowledge will easily transfer. That's one of the reasons why working with React can be such a pleasure.

There are many resources you can look into if you'd like to keep learning about React Native. One is *React Native in Action* by Nader Dabit (Manning Publications,

Licensed to Samir Mashlum <smashlum@gmail.com>

2018), shown in figure 13.5, which pairs nicely with this book because it allows you to pick up right where you leave off in learning React and is an excellent introduction to React Native. You'll apply your knowledge from your work in this book so far and use the momentum to dive into building mobile applications with React Native. It's also a good resource to look into next if your team is considering React Native for an upcoming project.

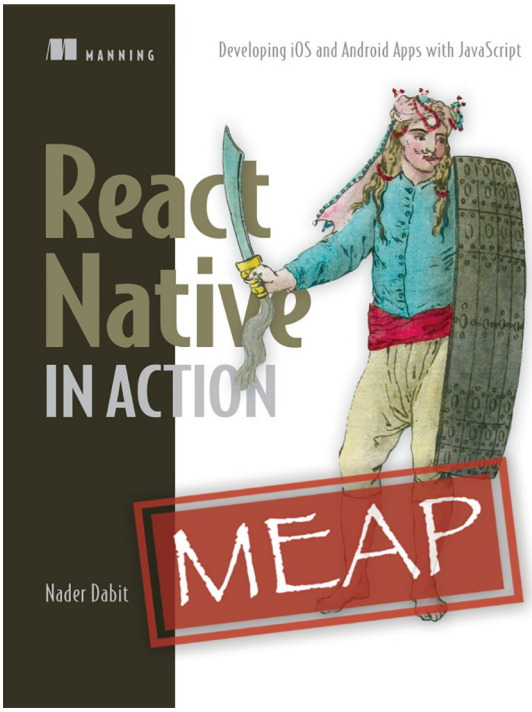


Figure 13.5 ***React Native in Action*** **by Nader Dabit gives iOS, Android, and web developers the skills they need to build robust, complex React Native applications. If you're still curious about React, it's the perfect book to transition to next. Learn more at www.manning.com/books/react-native-in-action.**

Another great resource to get you started with React Native is the Create React Native App project. Create React Native App provides an excellent starting place for a new React Native project or an excellent sample application for those just starting out with it. It includes a few preset libraries and tools for building React Native apps, but allows you to "eject" and reset to the default. If you're curious about Create React App or Create React Native App, check them out online:

- *Create React Native App*—https://github.com/react-community/create-react-native-app
- *Create React App*—https://github.com/facebook/create-react-app
- *React Native documentation*—https://facebook.github.io/react-native

Licensed to Samir Mashlum <smashlum@gmail.com>

13.6 Summary

Here's a recap of what you learned in this chapter:

- React Native is a technology in the React ecosystem that developers can use to write React applications that run on mobile iOS and Android devices.
- React Native uses the `React` core library for component creation but uses a different set of libraries to handle rendering your application on the native platform and to handle interactions with the underlying platform (touch events, geolocation, cameras access, and so on).
- React Native handles bridging between your JavaScript and the underlying mobile platform.
- React Native uses many APIs that are identical or similar to web APIs. It uses Flexbox for layout, Fetch for network requests, and other familiar APIs.
- You can mix JavaScript and native code when building React Native applications.
- React Native provides a robust set of tools for developing and compiling your applications.
- React Native's hot-reload developer tools save you time by not making you wait for your application to recompile every time.
- Using React Native can help lower the barrier to mobile development for you or your team.
- You won't want to use React Native for absolutely every type of mobile application, but it should be sufficient for most typical mobile applications.
- *React Native in Action* by Nader Dabit (Manning, 2018) is a great next resource to consider in your React journey—check it out at www.manning.com/books/react-native-in-action.

Licensed to Samir Mashlum <smashlum@gmail.com>